Name: Theophilus McEl Amafu
Student ID: 40276406
Email: theophilus.amafu@mail.concordia.ca
t_amafu@live.concordia.ca

Question 1:

Q1- given table :

		B_2		
		G	M	A
B_1	G	1	0	0
	M	0.1	0.9	0
	A	0.2	0.3	0.5

prior B_1 : $P(B_1 = G) = 0.90$
$P(B_1 = M) = 0.10$
$P(B_1 = A) = 0$

(a) $P(B_2 = G) = ?$
Law of Total probability :

$$P(B_2 = G) = P(B_2 = G \mid B_1 = G) P(B_1 = G) + P(B_2 = G \mid B_1 = M) P(B_1 = M) + P(B_2 = G \mid B_1 = A) P(B_1 = A)$$

$$P(B_2 = G) = (1)(0.90) + (0.1)(0.10) + (0.2)(0)$$
$$= 0.9 + 0.01 + 0$$
$$P(B_2 = G) = 0.91$$

i.e. the probability of B_2 to be Great band is 91%

(b) $P(B_1 = G \mid B_2 = G) = ?$

Bayes' Rule :

$$P(B_1 = G \mid B_2 = G) = \frac{P(B_2 = G \mid B_1 = G) P(B_1 = G)}{P(B_2 = G)}$$

Let:
$P(B_2 = G \mid B_1 = G) = 1$
$P(B_1 = G) = 0.90$
$P(B_2 = G) = 0.91$ (solved for in (a)).

$$P(B_1 = G \mid B_2 = G) = \frac{(1)(0.90)}{0.91}$$
$$= \frac{0.90}{0.91}$$
$$= 0.9890$$
$$P(B_1 = G \mid B_2 = G) = 98.90\%$$

Question 2: MLE and MAP

(a) MLE

$$L(\lambda) = \prod_{i=1}^{n} P(k_i \mid \lambda) = \prod_{i=1}^{n} \frac{\lambda^{k_i} e^{-\lambda}}{k_i!} = \frac{\lambda^{\sum_i k_i}\, e^{-n\lambda}}{\prod_i k_i!}$$

$$\ell(\lambda) = \ln L(\lambda) = \Big(\sum_{i=1}^{n} k_i\Big) \ln \lambda - n\lambda - \sum_{i=1}^{n} \ln(k_i!)$$

$$\frac{d\ell}{d\lambda} = \frac{\sum_{i=1}^{n} k_i}{\lambda} - n \;=\; 0$$

$$\frac{\sum_{i=1}^{n} k_i}{\lambda} = n$$

$$\frac{d^2\ell}{d\lambda^2} = -\frac{\sum_{i=1}^{n} k_i}{\lambda^2} < 0 \quad \text{(maximum)}$$

$$\boxed{\hat{\lambda}_{MLE} = \frac{1}{n} \sum_{i=1}^{n} k_i}$$

(b) MAP

$$\hat{\lambda}_{MAP} = \arg\max_{\lambda} P(\lambda \mid \mathcal{D}) = \arg\max_{\lambda} P(\mathcal{D} \mid \lambda)\, P(\lambda)$$

$$= \arg\max_{\lambda} \; \big[\ell(\lambda) + \ln P(\lambda)\big]$$

$$\ln P(\lambda) = (\alpha - 1)\ln\lambda - \frac{\lambda}{\beta} - \ln\Gamma(\alpha) - \alpha\ln\beta$$

$$J(\lambda) = \Big(\sum_{i=1}^{n} k_i\Big)\ln\lambda - n\lambda + (\alpha - 1)\ln\lambda - \frac{\lambda}{\beta} + \text{const}$$

$$= \Big(\sum_{i=1}^{n} k_i + \alpha - 1\Big)\ln\lambda - \Big(n + \frac{1}{\beta}\Big)\lambda + \text{const}$$

$$\frac{dJ}{d\lambda} = \frac{\sum_{i=1}^{n} k_i + \alpha - 1}{\lambda} - \Big(n + \frac{1}{\beta}\Big) = 0$$

$$\lambda\Big(n + \frac{1}{\beta}\Big) = \sum_{i=1}^{n} k_i + \alpha - 1$$

$$\frac{d^2 J}{d\lambda^2} = -\frac{\sum_{i=1}^{n} k_i + \alpha - 1}{\lambda^2} < 0 \quad (\alpha > 1, \text{ maximum})$$

$$\boxed{\hat{\lambda}_{MAP} = \frac{\sum_{i=1}^{n} k_i + \alpha - 1}{n + \frac{1}{\beta}}}$$

(c) Limit as $n \to \infty$

$$\hat{\lambda}_{MAP} = \frac{\sum_i k_i + \alpha - 1}{n + \frac{1}{\beta}} = \frac{\frac{1}{n}\sum_i k_i + \frac{\alpha - 1}{n}}{1 + \frac{1}{n\beta}}$$

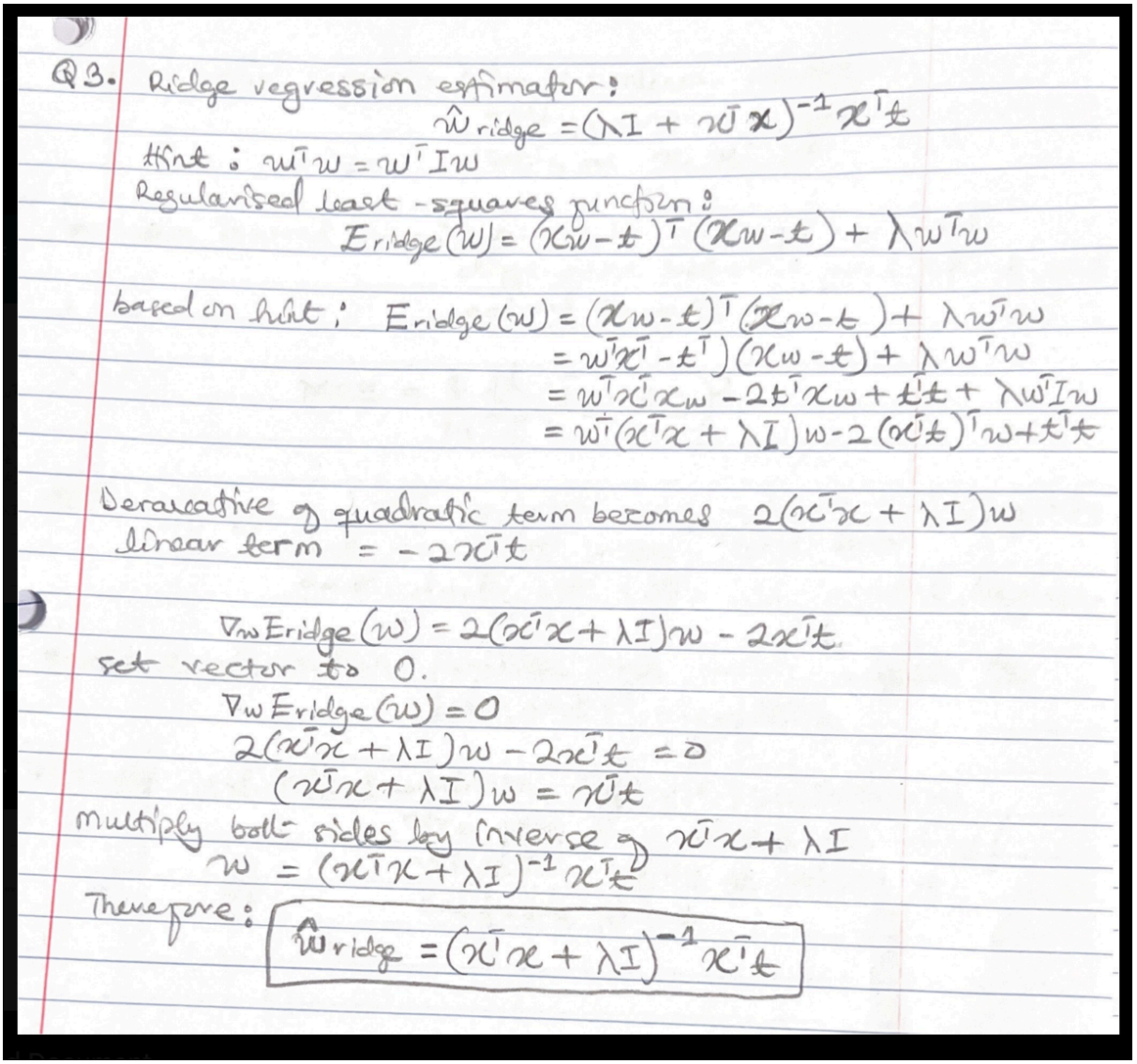
$$\frac{\alpha - 1}{n} \to 0, \qquad \frac{1}{n\beta} \to 0$$

$$\lim_{n\to\infty} \hat{\lambda}_{MAP} = \frac{\frac{1}{n}\sum_i k_i + 0}{1 + 0} = \frac{1}{n}\sum_i k_i = \hat{\lambda}_{MLE}$$

$$\hat{\lambda}_{MAP} - \hat{\lambda}_{MLE} \to 0$$

As n → ∞, λ̂MAP converges to λ̂MLE : the effect of the prior vanishes and both estimates equal the sample mean.

Question 3:



Q3. Ridge regression estimator:

$$\hat{w}_{ridge} = (\lambda I + x^T x)^{-1} x^T t$$

Hint: $w^T w = w^T I w$

Regularised least-squares function:

$$E_{ridge}(w) = (xw - t)^T (xw - t) + \lambda w^T w$$

based on hint: $E_{ridge}(w) = (xw - t)^T (xw - t) + \lambda w^T w$

$$= w^T x^T - t^T)(xw - t) + \lambda w^T w$$

$$= w^T x^T x w - 2t^T x w + t^T t + \lambda w^T I w$$

$$= w^T (x^T x + \lambda I) w - 2(x^T t)^T w + t^T t$$

Derivative of quadratic term becomes $2(x^T x + \lambda I) w$

linear term $= -2x^T t$

$$\nabla_w E_{ridge}(w) = 2(x^T x + \lambda I) w - 2x^T t$$

set vector to 0.

$$\nabla_w E_{ridge}(w) = 0$$

$$2(x^T x + \lambda I) w - 2x^T t = 0$$

$$(x^T x + \lambda I) w = x^T t$$

multiply both sides by inverse of $x^T x + \lambda I$

$$w = (x^T x + \lambda I)^{-1} x^T t$$

Therefore:

$$\hat{w}_{ridge} = (x^T x + \lambda I)^{-1} x^T t$$

Question 4: Programming: Nonlinear Regression

a) Figure 1. Training and test mean squared error (log scale) versus polynomial degree using all 7 features, with the first 100 points for training and the remaining points for testing, without regularization. Comment: The training error keeps decreasing as the polynomial degree increases, from about 0.18 at degree 1 to about 0.04 at degree

10. This is expected because higher-degree models have more parameters and can fit the training data better.However, the test error behaves differently. It decreases from about 0.19 at degree 1 to its lowest value of about 0.14 at degree 3. After that, it gradually increases until degree 7, then rises sharply at degree 8 to about 0.65. By degree 10, the test error reaches about 3.5, which is much higher than the training error.

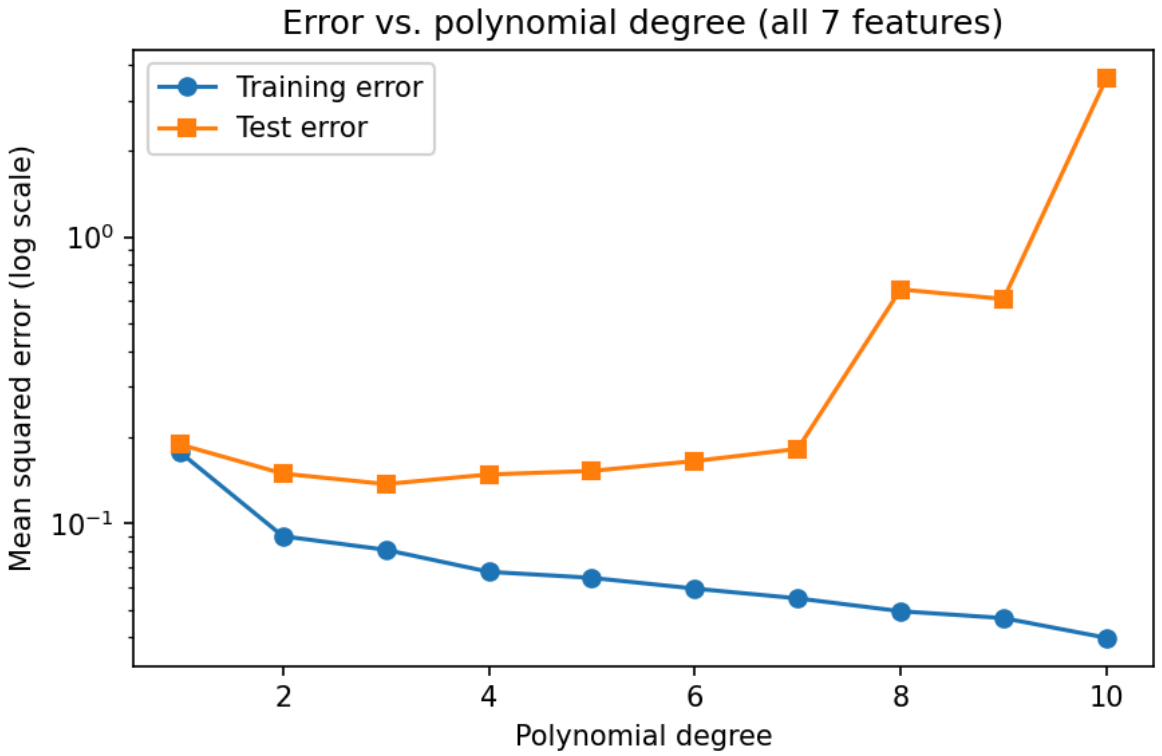


Figure 1. Training and test MSE vs. polynomial degree (log scale).

- **Underfitting** (degree 1): The model is too simple to capture the relationships in the data, resulting in high training and test errors.
- **Best generalization** (degrees 2–3): The test error is lowest around these degrees, meaning the model predicts unseen data better.
- **Overfitting** (degrees 8–10): The training error continues to decrease, but the test error increases significantly. With 7 features, a degree 8 model has 57 weights, including the bias, fitted using only 100 training points. Without regularization, the model can become too sensitive to the training data and produce unreliable predictions on new data.

b) Figure 2. Degree 2 fit (low order). Train MSE 0.348, test MSE 0.301.

The degree-2 polynomial is a smooth, gently curved line that follows the overall trend: MPG falls as feature 3 increases, then flattens out at high values. It passes through the middle of both the training and test points without chasing individual points. The train and test errors are close, and the test error is slightly lower (0.301 vs 0.348). This is a sign of good generalization, and the small gap probably reflects which points landed in each split. There is still a lot of scatter around the curve, so the remaining error is mostly noise in the data that a single feature cannot explain. The model slightly underfits the steep left end, where it sits a little below the highest points.

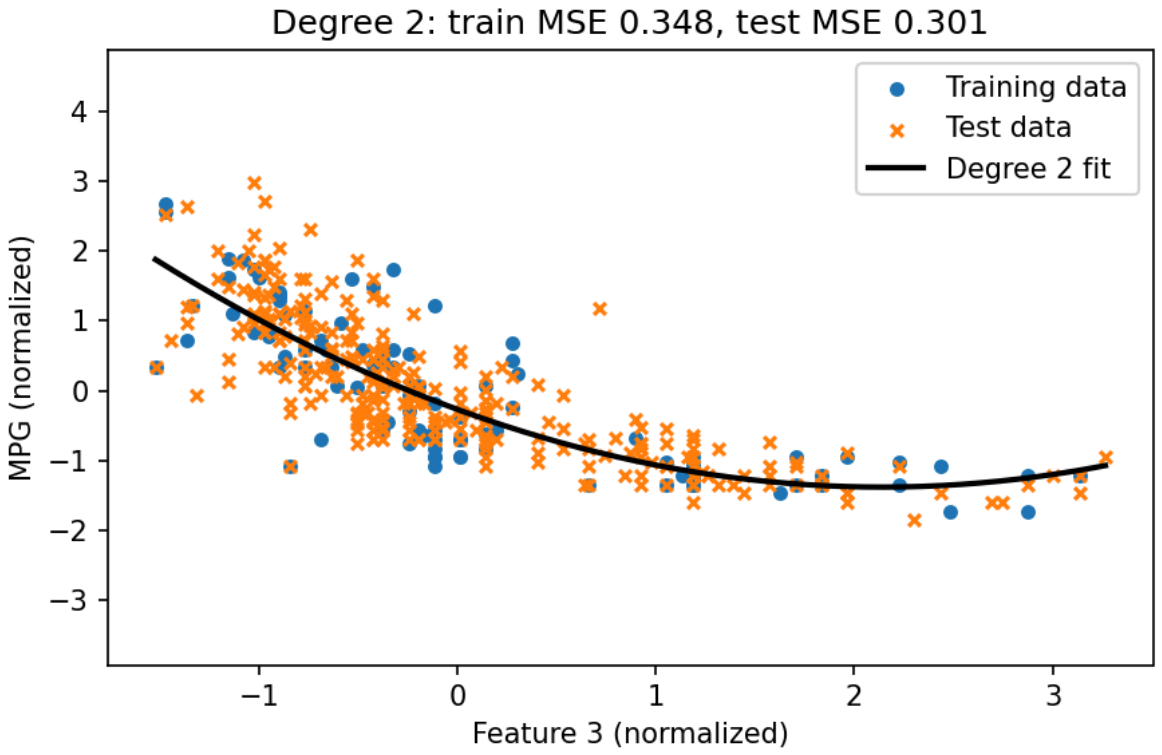


Figure 2. Degree 2 fit. Train MSE 0.348, test MSE 0.301.

Figure 3. Degree 10 fit (high order). Train MSE 0.327, test MSE 0.345.

The degree-10 curve is much more flexible. It wiggles through the dense cluster on the left, flattens and bumps in the middle, and oscillates near the right edge. The extra flexibility buys almost nothing on the training data (0.327 vs 0.348 for degree 2), and the test error is worse (0.345 vs 0.301), which is

the start of overfitting. The most visible problem is at the right edge, where the curve dives below −3.5 near feature 3 ≈ 3.2, even though the data there sit around −1. Where data are sparse, a high-degree polynomial is poorly constrained and extrapolates wildly. A few test points in that region make the test error clearly worse.

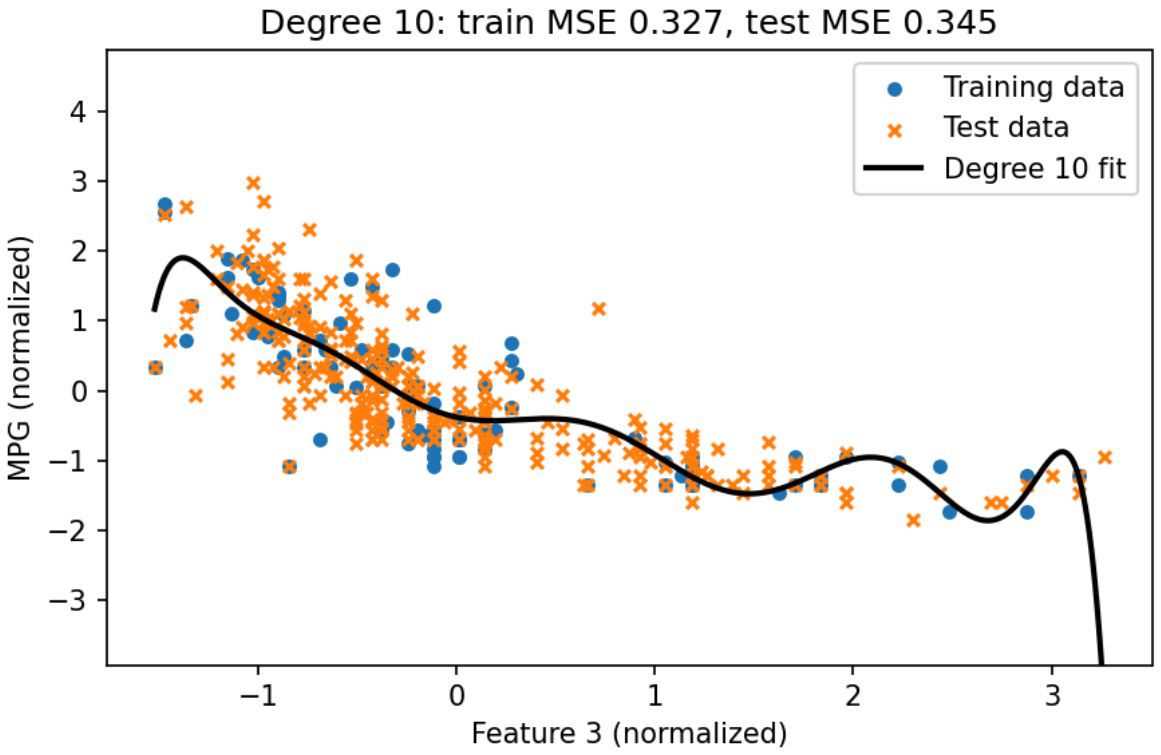


Figure 3. Degree 10 fit. Train MSE 0.327, test MSE 0.345.

c) polynomial_regression_reg.py

Validation error is about 0.51 with no regularization, falls to a minimum of about 0.43 at lambda = 1, and then rises (about 0.51 at lambda = 10, about 0.91 at lambda = 1000) as the penalty forces the weights toward zero and the model underfits.

Chosen regularizer: lambda = 1, the value with the lowest average validation error.

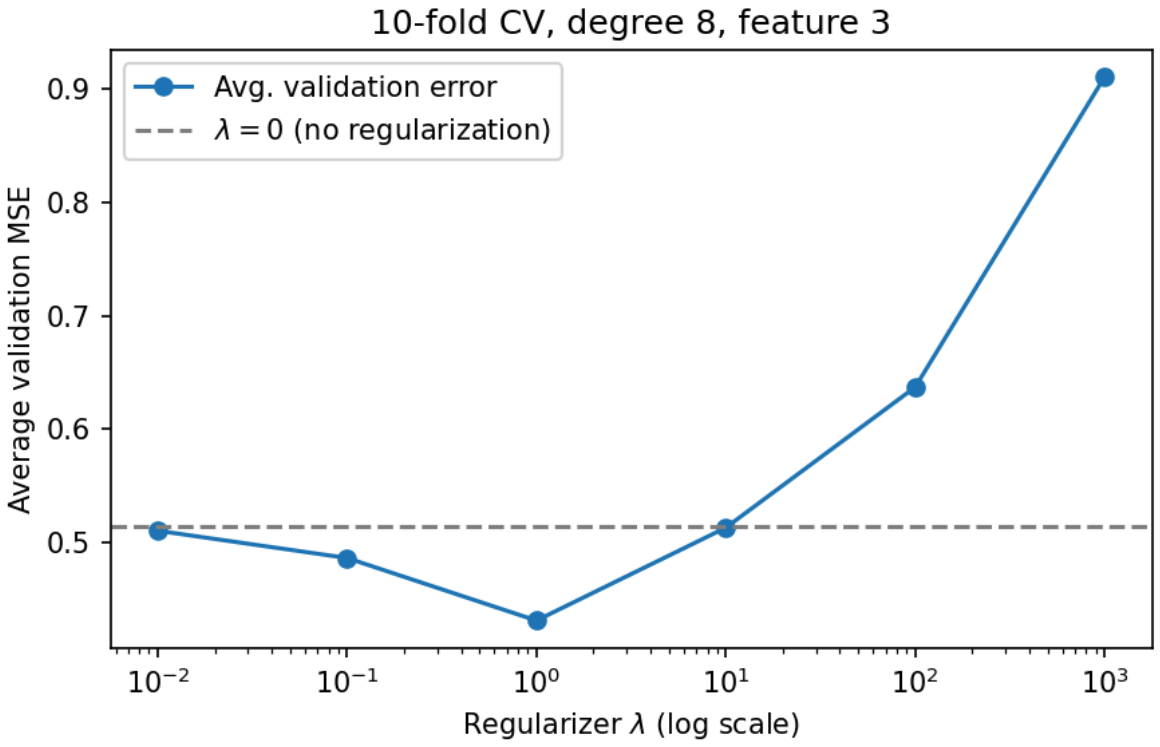


Figure 4. Average validation MSE vs. regularizer lambda (log scale). Dashed line: lambda = 0.

Reference: Code comment and organisation was done with help of ChatGpt and Claude. Also some part of the report was proofread, organised and summarised using the same AI tools.